

Abstract

Manga pages are difficult for computer vision systems to parse because their layouts are highly stylized, irregular, and often intentionally broken by artistic effects. This project addresses the problem of automatically identifying manga panels and associating speech bubbles with the correct panel regions when the layout cannot be inferred from complete borders or regular grids. We implement a multi-stage pipeline that combines image preprocessing, edge-based panel detection, morphological operations that reconnect broken panel-border segments, contour filtering, polygonal panel approximation, and speech-bubble detection. We evaluate the system using panel Intersection over Union, which measures how much a predicted panel overlaps with the correct ground-truth panel, panel precision and recall, panel count error, and speech-bubble detection accuracy. The system achieves **[insert main quantitative result]** on **[insert dataset/split]**. Our main contribution is a controlled analysis of how a hybrid structural and semantic pipeline performs across different categories of irregular manga layouts, including pages with incomplete borders, overlapping artwork, non-rectangular panels, and speech bubbles that cross panel boundaries.

1. Introduction

1.1 Why this problem exists

Manga pages contain highly stylized and irregular visual structures, including non-rectangular panels, speech bubbles, sound effects, and artwork that frequently extends beyond panel boundaries. Unlike word-processing documents or conventional web pages, manga does not follow a predictable grid or consistent reading structure. Characters, speech bubbles, and decorative effects may overlap multiple panels or interrupt panel borders entirely. This makes manga difficult for computer vision systems to parse because the page components are not always separated by clean lines or consistent spacing.

The research problem we define is manga panels parsing methods often break when the page structure cannot be inferred from complete borders, rectangular regions, or regular layouts. This task requires visual reasoning because the system must infer layout structure from visual evidence that is often incomplete or misleading. In a simple document, layout can often be recovered from whitespace, text blocks, or rectangular boundaries. In manga, those assumptions break down because the visual design is part of the storytelling. A panel border may be missing because a character breaks through it, or a speech bubble may sit between two panels while still belonging to only one

narrative moment. These cases make parsing difficult because the system must decide whether nearby visual elements should be grouped together, separated, or associated across a broken boundary.

1.2 Why solving it matters

This problem matters because many downstream applications depend on correctly identifying the individual components of a manga page. Automatic translation requires the system to know which speech bubbles belong to which panel before translating dialogue and make it audible in the correct order. A tool that reconstructs manga pages into editable digital layers also needs to separate panels, speech bubbles, and text regions before users can manipulate or reformat the page. More accessible digital readers could also benefit from this structure by presenting panels one at a time or supporting alternative reading formats.

The value of this project is not limited to manga alone. Manga is a useful test case for a broader computer vision problem: parsing visually complex documents whose structure cannot be recovered from regular grids or fully enclosed objects. Posters, advertisements, illustrated infographics, and other artistic documents can contain the same kinds of irregular text placement, overlapping graphics, and broken boundaries. Improving manga parsing therefore helps explore how computer vision systems can handle documents where artistic layout matters as much as textual content.

1.3 What this project evaluates

This project evaluates whether a hybrid pipeline combining geometric panel-boundary detection and speech-bubble detection can parse manga pages more robustly than methods based only on spatial proximity or simple border gaps. We compare different versions of the pipeline on annotated manga pages and measure how preprocessing, morphological border repair, contour filtering, and polygonal panel approximation affect panel detection.

We also evaluate whether speech-bubble detection improves the representation of page structure. Panel detection alone identifies the major regions of the page, but it does not fully describe how the page should be understood. Associating speech bubbles with panels moves the system closer to a useful page representation for downstream tasks such as translation, reading-order inference, and editable manga reconstruction.

2. Related Work

2.1 Layout analysis based on visual proximity

Prior work in document and webpage segmentation shows that visual layout can be useful for grouping page elements. Bajammal and Mesbah propose a webpage segmentation method that groups nearby visual elements into coherent regions based on visual adjacency [1]. Their approach is useful because it tries to capture how users visually perceive a page instead of relying only on the underlying document structure. This idea is relevant to our project because manga parsing also requires reasoning about what the page looks like, not just what object labels exist.

However, the assumption behind visual adjacency does not transfer cleanly to manga. Webpages usually have organized layouts where nearby elements often belong to the same section. Manga pages are different. Two adjacent regions may be separate panels, while objects belonging to the same narrative region may be separated by overlapping artwork or speech bubbles. A method based mainly on proximity may therefore merge panels that should remain separate or split one panel into multiple parts. This limitation motivates our choice to use visual structure, but not rely on adjacency alone.

2.2 Manga panel extraction based on border structure

Prior manga panel extraction work directly addresses the failure of simple edge-based methods by treating missing borders as part of the parsing problem. Ho et al. address the more directly related task of extracting panels from manga pages [2]. Their method improves on simpler connected-component and border-detection approaches by repairing incomplete panel borders and recursively splitting the page into panel regions. This is useful for our project because it shows that manga panel parsing cannot depend only on visible closed borders. Some borders must be inferred or repaired before the page structure can be recovered.

At the same time, their method still depends on the assumption that panels can be separated mainly through horizontal and vertical gaps. This works for many conventional layouts, but it becomes weaker when panels are circular, heavily overlapping, diagonal, nested, or interrupted by large foreground objects. In those cases, recursive splitting may divide one panel into multiple regions or merge multiple panels together. This is the main technical gap our project responds to: existing methods rely too much on either proximity or simple border structure, while manga often requires both geometric and semantic reasoning.

Our project's contribution will be a diagnostic evaluation of manga panel parsing on our diverged methodologies from prior work methods. We treat panel detection as a set of design choices—preprocessing, edge detection, border repair, contour filtering, and polygonal approximation—and test how each choice changes performance. This gives the project value even when the final method fails on difficult pages, because the results

show *why* it fails: for example, whether border repair helps incomplete panels but accidentally merges nearby panels, or whether contour filtering removes background noise but also removes small or unusual panels. Using Manga109 annotations for frames and other manga elements, we evaluate these stages across layout categories such as regular panels, irregular panels, incomplete-border panels, and overlapping-artwork pages. In this way, the project contributes a clearer experimental map of which classical computer vision assumptions are useful for manga panel parsing and which assumptions break under more stylized layouts.

2.3 How our method builds on prior work

Our method builds on both lines of work. From webpage layout segmentation, we take the idea that parsing should be based on visual structure rather than only document metadata. From manga panel extraction, we take the idea that incomplete borders must sometimes be repaired before panel regions can be recovered. This project is not a direct reimplementation of either related paper. We diverge in two ways. First, we implement the pipeline as a sequence of separable stages—preprocessing, edge detection, morphological repair, contour filtering, and polygonal approximation—so that each stage can be evaluated through ablation. Second, we analyze performance by manga layout category rather than only reporting an aggregate score. This allows us to test which assumptions fail on regular panels, irregular panels, incomplete borders, and overlapping artwork.

3. Methodology

3.1 Overview of the approach

The method separates manga parsing into structural and semantic stages. The structural stage detects panel boundaries: where panels begin and end, even when borders are broken or irregular. The semantic stage detects speech bubbles and assigns them to panel regions. This separation matters because a single visual cue is not enough for manga. Edges can identify borders, but they can also be confused by sound effects, hair, speed lines, or background textures. Speech-bubble detection can identify dialogue regions and combining the two stages gives the system more information than either stage alone about the page.

3.2 Dataset and annotation setup

We are using Manga109, which contains 21,142 pages in total; its annotations include page elements such as frames, text, faces, and bodies. We use it because it provides a large manga-specific dataset with annotations suitable for panel localization and

speech-bubble detection. We load the dataset by mapping each manga page image to its corresponding annotation file. The annotation loader extracts panel regions and speech-bubble regions, then converts them into internal formats used by the evaluation pipeline.

For panel detection, the annotations are converted into ground-truth panel regions so that predicted panel contours can be compared against them. For speech-bubble detection, annotations are converted into object-detection labels in YOLO format. Before training or evaluation, we generate visualizations of the ground-truth labels on top of the original manga pages. This sanity check is important because if annotations are misaligned, the later training and evaluation results become unreliable.

3.3 Preprocessing

The input to the system is a grayscale manga page. Many manga pages contain screentones, shading, background patterns, and smudging effects that are meaningful artistically but can interfere with panel detection. Since the panel detector focuses on strong line structure, the first step is preprocessing. We use binarization and noise reduction to emphasize panel borders and suppress background texture.

We compare simple global thresholding, Otsu thresholding, and adaptive thresholding on sample pages. A global threshold uses one cutoff for the whole image. Otsu thresholding automatically chooses a cutoff based on the image histogram. Adaptive thresholding chooses local cutoffs in different parts of the page, which can work better when shading or background intensity changes across the image. Morphological opening removes small noisy marks, while morphological closing connects small gaps in lines. The goal of preprocessing is not to perfectly simplify the image. It is to make the later edge and contour steps less sensitive to decorative textures.

3.4 Panel boundary detection

After preprocessing, the system detects candidate panel boundaries using edge-based methods. We apply Canny edge detection, which finds pixels where image intensity changes sharply. These sharp changes often correspond to borders, but in manga they can also come from hair, clothing folds, text, speed lines, and background effects. Canny therefore gives us a useful but noisy edge map rather than a finished panel detector.

Because Canny edges are often fragmented, we apply morphological operations such as dilation and closing to reconnect broken border segments. This step is related to the border-repair motivation in Ho et al.'s manga panel extraction method [2]. Dilation slightly expands edge pixels so nearby fragments can touch. Closing fills small gaps between edge segments. Together, these operations make panel borders more likely to form continuous panel outlines instead of disconnected line fragments.

Once the repaired edge map is produced, the system extracts contours, a continuous outline formed by connected edge pixels. In this project, contours are candidate panel shapes. Very small contours are removed because they usually correspond to text, texture, or decorative marks rather than full panels. Extremely thin or irregular contours are filtered using area, aspect ratio, and closure score. Closure score measures how close a contour is to forming a complete boundary. Since manga panels are not always rectangular, the system uses polygonal approximation instead of forcing every panel into a bounding box. This preserves irregular panel shapes and makes the representation closer to the actual page layout.

3.5 Speech-bubble detection and panel assignment

The second stage detects speech bubbles using YOLO, which stands for “You Only Look Once.” YOLO is an object detection model that predicts bounding boxes and object classes from an image in a single pass. The YOLO labels are generated from Manga109 annotations by converting each speech-bubble region into a normalized bounding box. During inference, the detector outputs predicted speech-bubble boxes for each manga page. These boxes are not the same as panel parsing by themselves; they are a second layer of information that helps describe how text regions relate to detected panels.

Detected speech bubbles are then assigned to panels. The default assignment rule is spatial overlap: each bubble is assigned to the panel with the highest overlap. This works when the bubble lies clearly inside one panel. However, manga often contains bubbles that touch or cross panel boundaries. For those cases, we also examine whether a tail-direction (speech bubble tail) heuristic can improve assignment. The heuristic estimates the direction of that tail and uses it as extra evidence when spatial overlap alone is ambiguous. The purpose is to connect the detected objects back to the panel structure instead of leaving them as isolated bounding boxes.

3.6 Hierarchical page representation

The final output of the system is a hierarchical page representation. Each detected panel becomes a region in the page. Speech bubbles are attached to the panel they most likely belong to. Panel adjacency relationships are also computed so that the system can support reading-order inference. This representation is more useful than a flat list of detections because downstream applications need relationships, not just boxes. A translation system, for example, needs to know both where a speech bubble is and which panel context it belongs to.

3.7 Evaluation metrics

We evaluate panel detection using Intersection over Union, or IoU, between predicted panel regions and ground-truth panel annotations. IoU measures how much the

predicted region overlaps the correct region from 0, no overlap, to 1, which is perfect overlap. We also measure panel precision and recall. A true positive is a predicted panel that correctly matches a ground-truth panel above the chosen IoU threshold. A false positive is a predicted panel that does not match any real panel. A false negative is a real panel that the system fails to detect. Precision measures how many predicted panels correspond to real panels, while recall measures how many ground-truth panels were successfully detected. Panel count error is also useful because manga panel parsing often fails by over-segmenting one panel into several pieces or under-segmenting several panels into one region.

3.8 Baselines and ablations

To avoid only reporting one final pipeline, we compare controlled versions of the method. For panel detection, the ablations are: no preprocessing, preprocessing only, Canny with contour extraction, Canny with morphological repair, Canny with geometric contour filtering, and the full panel pipeline with polygonal approximation. These versions isolate whether preprocessing, border repair, and contour filtering improve panel localization.

Speech-bubble detection and bubble-to-panel assignment are evaluated separately because they answer a different question. For speech bubbles, the controlled versions compare YOLO detection alone, YOLO with spatial-overlap panel assignment, and YOLO with spatial-overlap plus tail-direction assignment. This separation avoids confusing two tasks: panel parsing measures whether panels are detected correctly, while bubble association measures whether detected speech bubbles are connected to the correct panel.

These comparisons help explain why the results look the way they do. If morphological closing improves recall but increases false positives, that means it helps recover more real panels but also creates extra incorrect panel predictions. If contour filtering removes noise but accidentally removes small panels, that shows a tradeoff between removing false positives and preserving real but unusual panel shapes. These tradeoffs are central to the project because the goal is not only to report whether the final system works, but to understand which assumptions break on different manga layout styles.

3.9 Pipeline figure

Figure 1. Manga parsing pipeline. The system loads annotated manga pages, preprocesses each image, detects panel boundaries using edge and contour-based methods, detects speech bubbles using an object detector, and combines the outputs into a hierarchical page representation for evaluation.

4. Dataset and implementation setup

4.1 Dataset loading

The first implementation step is dataset loading rather than model training. We create a loader that maps each manga page image to its annotation file and prints basic dataset statistics, including the number of pages, panel annotations, and speech-bubble annotations. This step gives us a reliable view of the data before any algorithmic processing.

4.2 Annotation visualization

After loading the annotations, we create a visualization utility that draws panel and speech-bubble annotations on top of the original manga page. This utility is used throughout development to verify both ground-truth labels and predicted outputs. It also produces figures for the report, including examples of successful detections and failure cases.

4.3 Annotation conversion

The annotations are converted into two formats. Panel annotations are converted into polygon or mask representations for IoU evaluation. Speech-bubble annotations are converted into YOLO-format bounding boxes for object detection. This conversion makes it possible to evaluate panel segmentation and train speech-bubble detection inside the same project pipeline.

Individual Contributions- below is just temp holder

Jing Tan: Jing worked on [insert parts of code: e.g., dataset loading, preprocessing pipeline, panel boundary detection, visualization utilities, evaluation script]. Jing also wrote or drafted [insert writing sections: e.g., Introduction, Methodology, Results discussion, figures/captions] and contributed to the analysis of [insert analysis: e.g., failure cases, ablation results, ethical considerations].

Nivedha Srinivasan: Nivedha worked on [insert parts of code: e.g., Manga109 annotation parsing, YOLO label conversion, speech-bubble detection, panel-to-bubble assignment, qualitative visualization]. Nivedha also wrote or drafted [insert writing sections: e.g., Related Work, Dataset section, Experimental Results, Discussion/Limitations] and contributed to the analysis of [insert analysis: e.g., comparison to prior work, dataset limitations, final editing].

Joint work: Both team members contributed to project planning, debugging, interpretation of results, report revision, and final editing. Both reviewed the final report and agreed on the reported results, limitations, and contributions.